
ICD - INTRO. A CIENCIA DE DADOS E APRENDIZADO DE MAQUINA

Implementando Redes Neurais MLP em Python

Prof. Ramon Mayor Martins, Dr.

ramon.mayor at: ifsc.edu.br

Formas de Implementar uma Rede Neural MLP

Existem varios pacotes Python para criar MLPs. Cada um tem um nivel de controle e complexidade diferente.



Formas de Implementar uma Rede Neural MLP

Existem varios pacotes Python para criar MLPs. Cada um tem um nivel de controle e complexidade diferente.



scikit-learn

MLPClassifier / MLPRegressor

Basico

Uma linha cria a rede inteira. Ideal para prototipagem rapida e aprendizado. Pouco controle sobre detalhes.

```
MLPClassifier(  
    hidden_layer_sizes=(64,32),  
    activation='relu',  
    max_iter=200  
) .fit(X, y)
```



TensorFlow / Keras

Sequential + Dense

Intermediario

Monta camada por camada. Padrao da industria. Flexivel, com muitos recursos e comunidade enorme.

```
model = Sequential([  
    Dense(64, activation='relu'),  
    Dense(32, activation='relu'),  
    Dense(3, activation='softmax')  
])
```



PyTorch

nn.Module + nn.Linear

Avancado

Controle total. Define a rede como uma classe Python. Preferido em pesquisa academica.

```
class MLP(nn.Module):  
    def __init__(self):  
        self.fc1 = nn.Linear(4, 64)  
        self.fc2 = nn.Linear(64, 3)  
    def forward(self, x):  
        return  
        self.fc2(F.relu(self.fc1(x)))
```

Pipeline Comum a Todos os Pacotes

1 Carregar dados

```
from sklearn.datasets import load_iris
X, y = load_iris(return_X_y=True)
```

2 Dividir treino/teste

```
from sklearn.model_selection import
train_test_split
X_train, X_test, y_train, y_test = \
    train_test_split(X, y,
                    test_size=0.2)
```

3 Normalizar

```
from sklearn.preprocessing import
StandardScaler
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

4 Criar modelo

```
# Varia por pacote
# (proximos slides)
```

5 Treinar

```
model.fit(X_train, y_train)
```

6 Avaliar

```
from sklearn.metrics import
accuracy_score
y_pred = model.predict(X_test)
print(accuracy_score(y_test, y_pred))
```

Abordagem 1: scikit-learn (MLPClassifier)

Basico

pip install scikit-learn

sklearn_mlp.py

```
from sklearn.neural_network import MLPClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score

# 1. Carregar e dividir dados
X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# 2. Normalizar
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# 3. Criar e treinar (TUDO EM UMA LINHA!)
model = MLPClassifier(
    hidden_layer_sizes=(64, 32), # 2 camadas: 64 e 32 neuronios
    activation='relu',          # Funcao de ativacao
    solver='adam',              # Otimizador
    max_iter=200                # Numero maximo de epocas
)
model.fit(X_train, y_train)

# 4. Avaliar
y_pred = model.predict(X_test)
print(f"Acuracia: {accuracy_score(y_test, y_pred):.2%}")
```

Vantagens:

- Mais simples de todos
- Uma classe faz tudo
- Mesma API do KNN, SVM, etc.
- Bom para prototipos rapidos

Parametros principais:

- hidden_layer_sizes: tupla com neuronios por camada
- activation: 'relu', 'tanh', 'logistic'
- solver: 'adam', 'sgd', 'lbfgs'
- max_iter: numero de epocas

Limitacoes:

Sem GPU, sem callbacks, sem controle fino do treinamento.

Abordagem 2: TensorFlow/Keras (Sequential + Dense)

Intermediário

`pip install tensorflow`

keras_mlp.py

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelBinarizer

# 1. Carregar e preparar
X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
lb = LabelBinarizer() # One-hot encoding
y_train_oh = lb.fit_transform(y_train) # [0,1,2] -> [[1,0,0],...]
y_test_oh = lb.transform(y_test)

# 2. Construir modelo CAMADA POR CAMADA
model = Sequential([
    Dense(64, activation='relu', input_shape=(4,)),
    Dropout(0.2), # Regularizacao
    Dense(32, activation='relu'),
    Dense(3, activation='softmax') # Saida: 3 classes
])

# 3. Compilar
model.compile(optimizer='adam',
              loss='categorical_crossentropy', metrics=['accuracy'])

# 4. Treinar
history = model.fit(X_train, y_train_oh, epochs=100,
                    batch_size=16, validation_split=0.2)
```

Vantagens:

- Padrao da industria
- Controle camada por camada
- Suporte a GPU nativo
- Callbacks, early stopping, etc.
- Comunidade enorme

Elementos-chave:

- `Dense()`: camada totalmente conectada
- `Dropout()`: regularizacao
- `compile()`: define loss e otimizador
- `fit()`: treina (retorna history)

One-hot encoding:

Keras espera o target como vetor binario: classe 1
→ [0,1,0]

Keras: Entendendo o model.summary()

model.summary()

```
model.summary()

# Model: "sequential"
#
# Layer (type)      Output Shape      Param #
# =====
# dense (Dense)      (None, 64)        320
# dropout (Dropout)  (None, 64)        0
# dense_1 (Dense)    (None, 32)        2080
# dense_2 (Dense)    (None, 3)         99
# =====
# Total params: 2,499
# Trainable params: 2,499
```

Como ler:

dense (64): 4 entradas $\times$ 64 pesos + 64 bias = 320
parametros

dropout: 0 parametros (so desliga neuronios no
treino)

dense_1 (32): $64 \times 32 + 32 = 2080$

dense_2 (3): $32 \times 3 + 3 = 99$

Param = (entradas $\times$ neuronios) + bias

Plotar curvas de aprendizado

```
# Plotar curvas de aprendizado
import matplotlib.pyplot as plt

plt.plot(history.history['accuracy'], label='Treino')
plt.plot(history.history['val_accuracy'], label='Validacao')
plt.xlabel('Epocas')
plt.ylabel('Acuracia')
plt.legend()
plt.title('Curva de Aprendizado')
plt.show()
```

O objeto history guarda:

- loss e accuracy por epoca
 - val_loss e val_accuracy
- Util para diagnosticar overfitting!*

Abordagem 3: PyTorch (nn.Module)

Avancado

pip install torch

pytorch_mlp.py (parte 1: definicao)

```
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# 1. Preparar dados
X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
X_train = torch.FloatTensor(X_train) # Converter para tensores
y_train = torch.LongTensor(y_train)
X_test = torch.FloatTensor(X_test)
y_test = torch.LongTensor(y_test)

# 2. Definir a rede como uma CLASSE
class MLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(4, 64) # Camada 1
        self.fc2 = nn.Linear(64, 32) # Camada 2
        self.fc3 = nn.Linear(32, 3) # Saida
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(0.2)

    def forward(self, x): # Caminho dos dados
        x = self.dropout(self.relu(self.fc1(x)))
        x = self.relu(self.fc2(x))
        return self.fc3(x)
```

Elementos-chave:

- nn.Module: classe base para redes
- nn.Linear(in, out): camada densa
- forward(): define o fluxo dos dados

Diferença do Keras:

Voce define a estrutura (`__init__`) e o caminho dos dados (`forward`) separadamente.

Dados como tensores:

PyTorch usa Tensors, não arrays NumPy. Precisa converter:

- torch.FloatTensor(X) para features
- torch.LongTensor(y) para labels

PyTorch: Loop de Treinamento Manual

pytorch_mlp.py (parte 2: treinamento)

```
# 3. Criar modelo, loss e otimizador
model = MLP()
criterion = nn.CrossEntropyLoss()      # Funcao de perda
optimizer = optim.Adam(model.parameters(), lr=0.01)

# 4. Loop de treinamento (voce controla tudo!)
for epoch in range(100):
    model.train()                        # Modo treino
    optimizer.zero_grad()               # Zerar gradientes
    outputs = model(X_train)            # Forward pass
    loss = criterion(outputs, y_train)   # Calcular loss
    loss.backward()                     # Backward pass!
    optimizer.step()                   # Atualizar pesos
    if (epoch + 1) % 10 == 0:
        print(f"Epoca {epoch+1}, Loss: {loss.item():.4f}")

# 5. Avaliar
model.eval()                            # Modo avaliacao
with torch.no_grad():
    outputs = model(X_test)
    _, predicted = torch.max(outputs, 1)
    acc = (predicted == y_test).sum().item() / len(y_test)
    print(f"Acuracia: {acc:.2%}")
```

O loop explicita tudo:

1. `zero_grad()`

Limpa gradientes da iteracao anterior

2. `model(X_train)` → **Forward**

Dados passam pela rede

3. `criterion()` → **Calcular Loss**

Mede o erro

4. `loss.backward()` → **Backward**

Backpropagation! Calcula gradientes

5. `optimizer.step()` → **Atualizar**

Ajusta os pesos

Vantagem:

Controle total! Pode customizar cada etapa (ex: treinar so parte da rede, gradientes customizados).

Comparativo: Qual Pacote Usar?

Criterio	scikit-learn	Keras/TF	PyTorch
Complexidade	Basico	Intermediario	Avancado
Linhas de codigo	~5 linhas	~15 linhas	~30 linhas
Controle	Pouco	Medio	Total
GPU	Nao	Sim	Sim
Callbacks	Nao	Sim (built-in)	Manual
Loop de treino	Automatico	Automatico	Manual
Uso tipico	Prototipos, aulas	Producao, industria	Pesquisa, custom

Recomendacao para ADS: Comece com **scikit-learn** para entender o conceito, depois migre para **Keras** para ter mais controle. PyTorch se for para pesquisa ou TCC.

Resumo: Cheat Sheet de Implementacao

scikit-learn: MLPClassifier

```
# scikit-learn (5 linhas)
from sklearn.neural_network import MLPClassifier
model = MLPClassifier(hidden_layer_sizes=(64,32), max_iter=200)
model.fit(X_train, y_train)
print(model.score(X_test, y_test))
```

TensorFlow/Keras: Sequential + Dense

```
# Keras (10 linhas)
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
model = Sequential([Dense(64, activation='relu', input_shape=(4,)),
                    Dense(32, activation='relu'),
                    Dense(3, activation='softmax')])
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
model.fit(X_train, y_train_oh, epochs=100, batch_size=16)
```

PyTorch: nn.Module + loop manual

```
# PyTorch (loop explicit)
model = MLP()
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.01)
for epoch in range(100):
    optimizer.zero_grad()           # Zerar gradientes
    loss = criterion(model(X_train), y_train)  # Forward + Loss
    loss.backward()                  # Backward pass
    optimizer.step()                 # Atualizar pesos
```

Formas de Implementar uma Rede Neural MLP



O que é FastAI?

fast.ai

Making neural nets uncool again

Wrapper de alto nível sobre PyTorch

Criada por Jeremy Howard (curso fast.ai). Simplifica o PyTorch ao máximo, mantendo a flexibilidade.

Filosofia:

Fazer em 5 linhas o que levaria 30 em PyTorch puro. Defaults inteligentes que funcionam bem na maioria dos casos.

Ideal para:

- Dados tabulares (classificação/regressão)
- Imagens (CNNs com transfer learning)
- Texto (NLP, classificação de texto)
- Dados colaborativos (recomendação)

Onde o FastAI se encaixa?

FastAI

Alto nível, defaults inteligentes

PyTorch

Framework base, controle total

CUDA / GPU

Aceleração por hardware

vs scikit-learn: Mais poderoso (GPU, embeddings, lr_find)

vs Keras: Menos boilerplate, lr_find automático

vs PyTorch puro: Sem loop manual, tudo embutido

FastAI: MLP com Dados Tabulares (Iris)

Alto nível

`pip install fastai`

fastai_mlp_iris.py

```
from fastai.tabular.all import *
from sklearn.datasets import load_iris
import pandas as pd
# 1. Carregar dados como DataFrame
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['target'] = pd.Categorical(iris.target) # Categorico!

# 2. Criar DataLoaders (fastai prepara tudo!)
dls = TabularDataLoaders.from_df(
    df,
    cont_names=iris.feature_names,    # Features continuas
    y_names='target',                # Variavel alvo
    y_block=CategoryBlock(),          # Classificacao
    procs=[Normalize],               # Normalizar automaticamente
    valid_pct=0.2,                   # 20% validacao
    bs=32                             # Batch size
)

# 3. Criar learner (define a MLP automaticamente!)
learn = tabular_learner(
    dls,
    layers=[64, 32],                 # 2 hidden layers: 64 e 32 neuronios
    metrics=accuracy                  # Metrica de avaliacao
)

# 4. Encontrar melhor learning rate (recurso exclusivo!)
learn.lr_find()

# 5. Treinar
learn.fit_one_cycle(50, 1e-2)        # 50 epocas, lr=0.01
```

O que o FastAI faz por voce:

- Normalizacao automatica
- Split treino/validacao
- Batch loading otimizado
- Arquitetura MLP com BatchNorm e Dropout embutidos
- Learning rate finder
- One-cycle training policy

layers=[64, 32]:

Define 2 hidden layers com 64 e 32 neuronios. O fastai adiciona automaticamente BatchNorm e Dropout entre elas.

fit_one_cycle():

Treina com politica de learning rate que sobe e desce. Converte mais rapido que lr fixa.

FastAI: Recursos Exclusivos

lr_find() - Encontrar Learning Rate Ideal

Testa varias taxas de aprendizado e plota um grafico mostrando onde a loss cai mais rapido. Voce escolhe o ponto antes de explodir.

Nenhum outro framework faz isso automaticamente!

lr_find()

```
# Encontrar melhor LR
learn.lr_find()
# Retorna grafico com sugestao!

# Treinar com LR encontrado
learn.fit_one_cycle(50, 1e-2)
```

Predicao e Interpretacao

Fastai facilita predicao individual, em batch, e exportacao do modelo treinado para producao.

Predicao e exportacao

```
# Predicao individual
row, clas, probs = learn.predict(df.iloc[0])
print(f"Classe: {clas}, Probs: {probs}")

# Mostrar resultados
learn.show_results()

# Exportar modelo
learn.export('modelo_iris.pkl')

# Carregar depois
learn_loaded = load_learner('modelo_iris.pkl')
```

Comparativo: Mesmo Problema, 4 Abordagens

scikit-learn: MLPClassifier

```
# scikit-learn (4 linhas de modelo)
model = MLPClassifier(hidden_layer_sizes=(64,32), max_iter=200)
model.fit(X_train, y_train)
print(model.score(X_test, y_test))
```

TensorFlow/Keras: Sequential + Dense

```
# Keras (7 linhas de modelo)
model = Sequential([Dense(64, activation='relu', input_shape=(4,)),
                    Dense(32, activation='relu'),
                    Dense(3, activation='softmax')])
model.compile(optimizer='adam', loss='categorical_crossentropy')
model.fit(X_train, y_train_oh, epochs=100, batch_size=16)
```

FastAI: tabular_learner

```
# FastAI (3 linhas de modelo!)
dls = TabularDataLoaders.from_df(df, cont_names=features,
                                y_names='target', y_block=CategoryBlock(), procs=[Normalize])
learn = tabular_learner(dls, layers=[64, 32], metrics=accuracy)
learn.fit_one_cycle(50, 1e-2)
```

PyTorch: nn.Module + loop manual

```
# PyTorch (classe + loop manual)
model = MLP() # classe com nn.Linear(4,64), nn.Linear(64,32), nn.Linear(32,3)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.01)
for epoch in range(100):
    optimizer.zero_grad(); loss = criterion(model(X_train), y_train)
    loss.backward(); optimizer.step()
```


Quando Usar Cada Abordagem?

scikit-learn

~5 linhas

Prototipo rapido, aprender conceitos, dados pequenos/medios, mesma API dos outros algoritmos (KNN, SVM).

Keras

~15 linhas

Producao, controle por camada, GPU, callbacks, early stopping. Comunidade enorme.

FastAI

~5 linhas

Melhor dos dois mundos: simples como sklearn mas com GPU, lr_find, BatchNorm e Dropout automaticos.

PyTorch

~30 linhas

Controle total, pesquisa, TCC, customizacao de cada etapa do treinamento.

Sugestao para ADS: sklearn para aprender → **FastAI** para ir alem com pouco codigo → Keras/PyTorch para projetos mais complexos e TCCs.